



Time-Based One-Time Password TOTP

SEC3104 Advanced Cryptography

Team Members

Ghadah Fahad Al-Harhi 444006163
Reemas Abdulhakim Theban 444006605
Atheer Ahmad Al-Harhi 444006152
Sarah Ibrahim Al-Sulami 444002322

Project Group 35
April 22, 2026

Abstract

This project presents the design and implementation of a secure authentication system based on the Time-Based One-Time Password (TOTP) protocol. Traditional password based authentication is vulnerable to credential theft and phishing, making multi factor authentication (MFA) essential for modern systems. TOTP addresses these threats by generating short lived, time bound verification codes derived from a shared secret key and the current Unix time, ensuring that intercepted codes cannot be reused. The system is built on the HMAC-SHA-1 cryptographic primitive, producing 6 digit codes valid for 30 second windows. The implementation, developed in Python using standard library modules, models the full authentication lifecycle across three phases: enrollment, OTP generation, and verification.

Keywords: Time-Based One-Time Password (TOTP), HMAC-SHA-1, Authentication, Multi Factor Authentication (MFA)

1 Introduction

The rapid digitalization of services has fundamentally transformed how individuals and organizations manage sensitive information. However, this transformation has simultaneously expanded the attack surface available to cybercriminals, resulting in increasingly frequent and costly security breaches. Traditional single factor authentication relying solely on static passwords has proven insufficient against modern threats such as phishing, brute force attacks, and credential stuffing [1]. Static passwords are inherently vulnerable; they can be guessed, intercepted, or stolen, and once compromised, they grant persistent unauthorized access. To address these limitations, Multi Factor Authentication (MFA) has emerged as a critical security enhancement. Among MFA mechanisms, the Time-Based One-Time Password (TOTP) protocol standardized by the Internet Engineering Task Force (IETF) in RFC 6238 offers a compelling solution [2]. TOTP generates short lived numeric codes by combining a shared secret key with the current time, producing codes that typically expire every 30 second. This time bound nature renders intercepted codes useless, effectively mitigating replay attacks and significantly reducing the risk of unauthorized access [3].

1.1 Real-World Scenario

In online banking systems, users authenticate using a password as the first factor. However, passwords can be compromised through phishing or data breaches. To address this, banks implement multi factor authentication mechanisms such as Time-Based One-Time Passwords (TOTP), where a user must provide a temporary code generated on a personal device. These codes typically refresh every 30 seconds and are required in addition to the password. Even if an attacker obtains the user's credentials, they cannot gain access without the second authentication factor, as MFA requires possession of an additional authenticator. This approach significantly reduces the risk of unauthorized access and is widely used in real world financial systems.

1.2 Why Cryptography is Needed

Cryptography is fundamental to securing information systems, as it provides mechanisms to protect data from cybercriminals and unauthorized access, ensuring that sensitive information reaches only its intended recipient. By enabling confidentiality, integrity, and authentication, cryptography supports secure communication and transactions over public networks. It achieves this by transforming plaintext into unreadable ciphertext and by verifying the authenticity of transmitted data using cryptographic primitives such as hash functions and message authentication codes [4]. However, traditional password based authentication remains insufficient against modern threats. Static credentials are vulnerable to phishing, brute force attacks, and credential reuse, especially once they are compromised. Cryptographic mechanisms address these limitations by enabling more secure authentication methods. In particular, HMAC-based one-time password systems use one way hash functions to verify authenticity without exposing sensitive information, binding each authentication token to a shared secret timestamp [3].

1.3 Objectives

This project aims to design and implement a custom TOTP based authentication protocol for securing electronic payment systems. The specific objectives are:

1. To design a two factor authentication protocol combining static passwords with time based one time codes.
2. To implement a server side verification mechanism that generates and validates TOTP codes without storing them.

3. To demonstrate correct and incorrect authentication scenarios through a command line prototype.

1.4 Scope

The scope of this project is limited to the cryptographic design and simulation of the authentication layer. The prototype will be implemented as a command line application simulating the client server interaction on a single machine. The implementation will focus on the correct application of HMAC-SHA1, time step computation, and code verification as specified in RFC 6238, using Python and its well audited cryptography libraries.

2 Requirements Analysis

2.1 Functional Requirements

- **FR-001 User Login and Credential Verification**
The system shall allow users to log in using a username and password and verify the credentials before initiating the TOTP authentication process.
- **FR-002 TOTP Generation**
The system shall generate a Time-Based One-Time Password (TOTP) using a shared secret key and the current Unix time.
- **FR-003 OTP Verification**
The system shall validate the user provided OTP by comparing it with the generated TOTP using the shared secret key.

2.2 Security Requirements

- **SR-001 Secret Key Management**
The system shall securely generate and use the shared secret key only during runtime for OTP generation and verification.[5, 6].
- **SR-002 OTP Expiration Enforcement**
The system shall ensure that each OTP is valid only for a fixed time window (30 seconds) and reject expired OTPs[5].
- **SR-003 Replay Attack Prevention**
The system shall ensure that each OTP can be used only once and reject any attempt to reuse previously validated OTPs[5, 6].
- **SR-004 Brute Force Protection**
The system shall limit the number of failed OTP verification attempts to prevent brute-force attacks [6].

2.3 Assumptions

- System time is accurate and synchronized.
- Shared secret key is securely stored and accessed.
- Cryptographic primitives (e.g., HMAC-SHA1) are correctly implemented.

3 Proposed Protocol Design

3.1 Step-by-Step Description

Time-based One-Time Password (TOTP) is an algorithm that generates a one time password using a shared secret key and the current time. In our protocol, the user first logs in with a username and password. After successful verification, the server requests a TOTP to complete authentication. The TOTP is generated locally on the user's device and verified by the server, which independently computes the expected value using the same shared key and time reference. If the values match, access is granted[7]. Both client and server rely on a synchronized time source based on Unix time (seconds since January 1, 1970, UTC) and use a fixed time step of 30 seconds. The algorithm is based on HOTP and requires secure handling of the shared secret key for correct verification.

3.2 Sequence Diagram

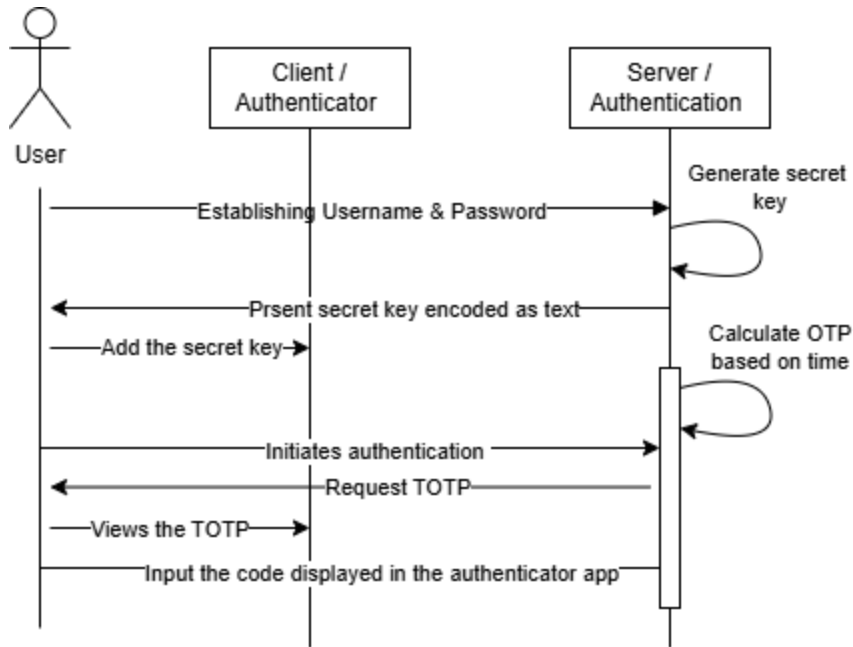


Figure 1: TOTP Sequence Diagram.

3.3 Justification of Cryptographic Primitives

3.3.1 Cryptographic Foundation

The use of HMAC (Hash-based Message Authentication Code) in TOTP is a deliberate architectural requirement mandated by RFC 6238[5] and RFC 4226[8], grounded in the mathematical properties defined in RFC 2104[9]. The foundational computations are defined as:

- **HOTP:**

$$HOTP(K, C) = Truncate(HMAC - SHA - 1(K, C)) \quad (1)$$

- **TOTP:**

$$TOTP(K, T) = HOTP(K, T) \quad (2)$$

HMAC uses a two pass nested construction to ensure security:

$$HMAC(K, T) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel T)) \quad (3)$$

3.3.2 Security Properties of HMAC

HMAC offers security properties worth highlighting. First, it remains secure even when the underlying hash function has known collision vulnerabilities[9]. Second, HMAC-based authentication requires a unique shared secret K for each user, as specified in RFC 6238[5], which ensures that observing one user's OTP provides no advantage in predicting future codes or compromising other users' credentials.

3.3.3 30 Second Time Step Justification

RFC 6238 explicitly frames the selection of a 30 second time step as a trade off between security and usability. It states that this default value was chosen to balance the need for strong security with practical usability in real world authentication systems.

where is this in your references?

3.3.4 6-Digit OTP Justification

According to RFC 4226, the HOTP value must be at least 6 digits to resist brute force attacks by providing 1,000,000 possible values while still being easy to read, enter, and compatible with limited devices. The probability of correctly guessing an OTP is given by:

$$P(\text{correct guess}) = \frac{1}{10^d}$$

For a 6 digit OTP, this becomes:

$$P = \frac{1}{10^6}$$

When considering authentication attempts with throttling (T attempts), the probability of success is bounded by:

$$P(\text{success}) \leq \frac{T}{10^d}$$

For example, if $T = 3$:

$$\frac{3}{10^6} = 0.0003\%$$

4 Implementation Plan

4.1 Language and Libraries

The system is implemented in Python using standard library modules like hmac and hashlib for the HMAC-SHA-1 construction, time for Unix epoch resolution, and secrets for cryptographically random key generation.

4.2 Simulation Approach

There is 3 phases of TOTP, witch is enrollment, OTP generation, and verification using two components: a Prover (client) and a Verifier (server). Enrollment generates a 160 bit shared secret distributed to both sides. Generation follows the RFC 6238 computation chain: time step T , HMAC-SHA-1, and dynamic truncation. Verification checks across a ± 1 time step window, enforces one time use, and applies a failed attempt lockout.

4.3 Key Algorithms

The TOTP algorithm is a version of the HOTP algorithm.[10] .This algorithm also depends on the shared secret key and the moving factor ut the process of generating moving factors is different. At TOTP moving factor will continue to change depending on time generate. The calculation process is the same as HOTP. The general formula is mentioned previously in the equation2. Where K is the shared secret key and T is an integer value that represents time. T is calculated using the function:

$$T = \frac{CurrentTime - T_0}{X} \quad (4)$$

Tcurrent is the current time in seconds whereas T_0 is agreed upon initial initiation, usually the value is set to zero ($T_0 = 0$). The parameter X determines how long a TOTP is valid. The standard used is 30. So by filling in $X = 30$, we allow one OTP is only valid for 30 seconds[11].

5 Team Roles

Team Member	Primary Responsibilities
Ghadah	Proposed Protocol Design, Related Work, System Design, Implementation, Discussion
Atheer	Introduction, Team Roles, Timeline, Background, Related Work, System Design, Implementation
Sarah	Abstract, Implementation Plan, Related Work, System Design, Implementation, Conclusion and Future Work
Reemas	Requirements Analysis, Related Work, System Design, Implementation, Evaluation

Table 1:

6 Timeline

Week	Milestone
Week 11	Background Related Work
Week 12	Secret Key + TOTP Generator
Week 13	Registration + Authentication Flow
Week 14	Security Testing
Week 15	Write the Report Paper
Week 16	Submission

Table 2:

References

- [1] S. Mishra, K. Meenakshi, and D. Bhargava, “Multi-factor authentication using time-based one-time password: A comprehensive analysis of security enhancement and implementation strategies,” pp. 100–105, 2025.
- [2] D. M’Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, “TOTP: Time-Based One-Time Password Algorithm,” Internet Engineering Task Force (IETF), Tech. Rep. RFC 6238, 2011. [Online]. Available: <https://doi.org/10.17487/RFC6238>
- [3] M. A. Hassan, Z. Shukur, and M. K. Hasan, “An improved time-based one time password authentication framework for electronic payments,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, no. 11, pp. 359–366, 2020. [Online]. Available: <https://doi.org/10.14569/IJACSA.2020.0111145>
- [4] M. T. Gençoğlu, “Importance of cryptography in information security,” *IOSR Journal of Computer Engineering (IOSR-JCE)*, vol. 21, no. 1, pp. 65–68, 2019.
- [5] IETF, “Totp: Time-based one-time password algorithm,” <https://datatracker.ietf.org/doc/html/rfc6238>, 2011.
- [6] N. I. of Standards and Technology, “Digital identity guidelines: Authentication and lifecycle management,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep. SP 800-63B, 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63b.pdf>
- [7] C. Sudar, S. K. Arjun, and L. R. Deepthi, “Time-based one-time password for wi-fi authentication and security,” in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, 2017, pp. 1212–1216.
- [8] IETF, “Hotp: An hmac-based one-time password algorithm,” <https://datatracker.ietf.org/doc/html/rfc4226>, 2005.
- [9] —, “Hmac: Keyed-hashing for message authentication,” <https://datatracker.ietf.org/doc/html/rfc2104>, 1997.
- [10] L. Lumburovska, J. Dobрева, S. Andonov, H. M. Trpcheska, and V. Dimitrova, “A comparative analysis of hotp and totp authentication algorithms. which one to choose?” *International Scientific Journal "Security & Future"*, vol. 5, no. 4, pp. 131–136, 2021.
- [11] H. Seta, T. Wati, and I. C. Kusuma, “Implement time based one time password and secure hash algorithm 1 for security of website login authentication,” in *2019 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, 2019, pp. 115–120.